أَعُوذُ بِكَلِمَاتِ اللَّهِ التَّامَّةِ مِنْ كُلِّ شَيْطَانٍ وَهَامَّةٍ، وَمِنْ كُلِّ عَيْنٍ لَامَّةٍ

secure notes vault

A'udhu bi kalimatillahit tammati min kulli shaitanin wa hammatin,
wa min kulli ainin lammah.

Opening dua: I seek refuge in the perfect words of Allah from every devil and every harmful creature, and from every envious eye.

Complete Project Documentation

Made by Team BreakByte

Live site: https://raito-del.github.io/Dead-Drop/

Repository: github.com/raito-del/Dead-Drop

Document date: 22 October 2026

Contains no passwords, keys, tokens or email addresses.

Contents

1. Overview

2. Current features

3. Recent changes in this version

4. Technology and architecture

5. File structure

6. Firebase setup

7. GitHub Pages and access token

8. Admin guide

9. Theme and design

10. Security notes and limitations

11. Rebuild steps

Appendix A. Complete source code (files A to J)

1. Overview

Dead-Drop is a password-protected notes vault made by Team BreakByte. Members open the site, enter an access key, and browse study notes organized in folders. The site owner logs in as admin to create folders and upload PDFs directly from the website, without using the GitHub web editor.

The site is a static website: plain HTML, CSS and JavaScript modules. Login uses Firebase Authentication. Notes are stored as files in a GitHub repository and served through GitHub Pages.

2. Current features

Area	Feature
Access	Animated terminal intro, then password (viewer account) to enter
Access	Admin login from the notes page with a separate admin password
Notes	Folder view with Open buttons and breadcrumb navigation
Notes	Search across all folders by file name or path
Notes	Read button opens the PDF inside the site (PDF viewer panel)
Notes	Download button saves the PDF to the device
Admin	Save a GitHub token once in the browser, or delete it
Admin	Create folders, including nested folders
Admin	Upload a PDF to the current folder, with an optional display name
Design	Matrix background, hacker theme, Arabic and Bangla footer quote

3. Recent changes in this version

- All visible text and code comments are in English.
- Footer on the home page and notes page shows the opening dua in Arabic and Bangla, with a team line below it.
- Notes page now uses folders with breadcrumb navigation instead of a flat list.
- Admin panel: create folders, upload PDFs into the current folder, save or delete the GitHub token in the browser.
- Search works across all folders and shows the folder path under each result.
- Each PDF has a Read button (in-site viewer) and a Download button.
- style.css includes the footer styles, breadcrumb styles, and a small-screen rule for the Arabic text.
- config.js holds the site settings. Real Firebase values and email addresses are kept private, not in this document.

4. Technology and architecture

Layer	Technology
Pages	Plain HTML: index.html (home and login), notes.html (notes and admin)
Styling	One stylesheet, style.css, with theme variables in :root
Logic	Vanilla JavaScript ES modules, no build step
Login	Firebase Authentication (Email/Password) from the gstatic CDN
Storage	PDFs and folders in the GitHub repository under notes/

Layer	Technology
Listing	GitHub REST API (git trees) lists every file and folder
Upload	GitHub REST API (contents) creates folders and commits PDFs
Hosting	GitHub Pages from branch main, root folder
Viewer	Mozilla pdf.js web viewer, loaded from mozilla.github.io

Data flow: the notes page reads the repository tree from GitHub, builds the folder view in the browser, and links each Download to the raw file URL. Admin actions write new files back to the repository through the GitHub API, using the token saved in that browser.

5. File structure

File	Purpose
index.html	Home page: typing intro, access key box, footer quote
notes.html	Notes page: folder list, search, admin login, admin panel, footer quote
style.css	Theme variables, layout, buttons, footer and responsive rules
config.js	Site name, GitHub repo details, Firebase settings, account emails
auth.js	Firebase login, logout, login state, admin check, error messages
github.js	Token storage, folder creation, PDF upload to GitHub
notes.js	Notes page logic: load files, folder view, search, viewer, admin panel
matrix.js	Matrix rain background on the canvas
typing.js	Typing animation on the home page, then shows the access box
README.md	Short project description for visitors
notes/	Folder that stores all PDFs, created from the site or by upload

6. Firebase setup

Firebase is used only for login. Follow these steps once:

1. Create a Firebase project and add a Web app. Copy the six config values into config.js, in the matching fields.
2. In Authentication, enable Email/Password sign-in.
3. Create two users: a shared viewer account (its email goes in VIEWER_EMAIL) and your admin account (its email goes in ADMIN_EMAIL). Use strong passwords and do not write them in any code file.
4. In Authentication settings, add your GitHub Pages domain (raito-del.github.io) to Authorized domains.

7. GitHub Pages and access token

1. Create a public repository and upload the files. Notes must be in the notes/ folder.
2. In Settings, Pages, choose branch main and folder root, then save.
3. Create a GitHub fine-grained token: Only select repositories, choose this repository, and give Contents permission Read and write. Metadata Read-only is added automatically.
4. Save the token once in the admin panel. It is kept only in that browser. Never paste it into code, commits, chat or this document.

Public repository means the PDFs and the file list are visible to anyone who finds the repository. The site password only protects the front door.

8. Admin guide

- Open the notes page, click Admin, enter the admin password. The admin panel appears under the notes.
- Save the GitHub token once. The panel then shows Delete token.
- Open a folder, or stay at the top, then type a name in New folder and click Create folder.
- Open a folder, select a PDF, optionally type a display name, and click Upload PDF.
- Use the search box to find any PDF across all folders.
- After an upload, the list refreshes automatically. GitHub may take a minute to publish changes to Pages.

9. Theme and design

Variable	Value	Used for
--bg	#050805	Page background
--panel	rgba(8, 16, 8, 0.85)	Boxes and footer
--text	#b6ffb6	Body text
--muted	#5f8f5f	Hints, footer, meta text
--accent	#00ff66	Headings, buttons, glow
--danger	#ff4d4d	Error text
--border	#12401f	Borders and dividers
--font	Consolas, Courier New, monospace	All text

To change the whole look, edit only the values inside :root in style.css. The matrix background is drawn by matrix.js with the same green.

10. Security notes and limitations

- Firebase web config values are not secret by design, but this project keeps them out of published documents anyway.
- Viewer and admin logins depend on Firebase Authentication. Keep the admin password private.
- The GitHub token has write access to this repository. Keep it in the browser only, and delete it on shared computers.
- Anyone with the repository link can read the PDFs, because GitHub Pages needs a public repository.
- The Read viewer uses an external Mozilla web page. If it is blocked, the Download button still works.
- The site uses a shared viewer password. Change it if it spreads beyond the group.

11. Rebuild steps

1. Create a public GitHub repository and a notes/ folder.
2. Create the files listed in section 5, then copy the contents from Appendix A into them, file by file, with the same names.
3. Edit config.js with your own GitHub username, Firebase values and account emails.
4. Set up Firebase (section 6) and GitHub Pages with a token (section 7).
5. Test locally with VS Code Live Server, then push with GitHub Desktop and check the live site.

Appendix A. Complete source code

Each file starts on a new page. Lines longer than the page width are wrapped, with the wrapped part indented. Characters the PDF fonts cannot draw (Arabic, Bangla, Japanese katakana) appear as [...]: copy those parts from the source file. config.js uses placeholders: replace them with your own values.

A.A index.html (part 1 of 2)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dead-Drop</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <canvas id="matrix"></canvas>

  <main class="wrap center">
    <div class="terminal">
      <div class="terminal-bar">root@dead-drop:~$</div>

      <h1>DEAD-DROP</h1>
      <div class="hint">secure notes vault</div>
      <br>

      <div id="typing-lines"></div>

      <!-- Password box: typing.js shows this once the typing animation finishes -->
      <form id="gate" class="hidden" autocomplete="off">
        <div class="row">
          <input type="password" id="pw" placeholder="enter access key" required>
          <button type="submit" class="btn" id="go">Unlock</button>
        </div>
        <div class="error" id="msg"></div>
      </form>
    </div>
  </main>

  <footer class="site-footer">
    <p class="quote-ar" lang="ar" dir="rtl">[...] [...] [...] [...] [...] [...] [...] [...] [...] [...]
        [...] [...]</p>
    <p class="quote-bn">[...] [...] [...] [...] [...] [...] [...] [...] [...], [...] [...] [...] [...]
        [...]</p>
    <div>Made by Team <b>BreakByte</b></div>
  </footer>

  <script type="module" src="matrix.js"></script>
  <script type="module" src="typing.js"></script>
  <script type="module">
    import { loginViewer, watchAuth, errorText } from "./auth.js";

    const form = document.getElementById("gate");
    const pw = document.getElementById("pw");
    const msg = document.getElementById("msg");
    const btn = document.getElementById("go");

    // If already logged in, go straight to the notes page
    watchAuth((user) => {
      if (user) location.href = "notes.html";
    });

    form.addEventListener("submit", async (e) => {
      e.preventDefault();
      msg.textContent = "";
      btn.disabled = true;
      btn.textContent = "Checking...";
      try {
        await loginViewer(pw.value);
        location.href = "notes.html";
      } catch (err) {
        msg.textContent = errorText(err);
        pw.value = "";
```

A.A index.html (part 2 of 2)

```
        btn.disabled = false;
        btn.textContent = "Unlock";
      }
    });
  </script>
</body>
</html>
```

A.B notes.html (part 1 of 2)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dead-Drop | Notes</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <canvas id="matrix"></canvas>

  <main class="wrap">
    <!-- If not logged in, notes.js will not show this -->
    <div id="content" class="hidden">

      <div class="topbar">
        <h2>root@dead-drop:~/notes$</h2>
        <div class="row" style="margin-top:0">
          <button class="btn" id="admin-toggle">Admin</button>
          <button class="btn" id="logout">Logout</button>
        </div>
      </div>

      <div class="terminal">
        <input type="search" id="search" placeholder="search notes...">
        <div class="hint" id="status"></div>
        <div class="list" id="list"></div>
      </div>

      <!-- Admin login box: appears when the Admin button is clicked -->
      <form id="admin-login" class="admin-panel hidden" autocomplete="off">
        <h2>admin access</h2>
        <div class="row">
          <input type="password" id="admin-pw" placeholder="admin password" required>
          <button type="submit" class="btn">Login</button>
        </div>
        <div class="error" id="admin-msg"></div>
      </form>

      <!-- Upload panel: only visible after admin login -->
      <div id="admin-panel" class="admin-panel hidden">
        <h2>admin: upload pdf</h2>
        <div class="hint">Drag the PDF onto the GitHub page, then click Commit changes.</div>
        <div class="row">
          <button class="btn" id="upload-btn">Open GitHub Upload</button>
        </div>
      </div>

    </div>
  </main>

  <footer class="site-footer">
    <p class="quote-ar" lang="ar" dir="rtl">[...] [...] [...] [...] [...] [...] [...] [...] [...] [...]
        [...] [...]</p>
    <p class="quote-bn">[...] [...] [...] [...] [...] [...] [...] [...] [...], [...] [...] [...] [...]
        [...]</p>
    <div>Made by Team <b>BreakByte</b></div>
  </footer>

  <script type="module" src="matrix.js"></script>
  <script type="module" src="notes.js"></script>
  <script type="module">
    import { loginAdmin, errorText } from "./auth.js";

    const toggle = document.getElementById("admin-toggle");
    const form = document.getElementById("admin-login");
```

A.B notes.html (part 2 of 2)

```
    const pw = document.getElementById("admin-pw");
    const msg = document.getElementById("admin-msg");

    toggle.addEventListener("click", () => form.classList.toggle("hidden"));

    form.addEventListener("submit", async (e) => {
      e.preventDefault();
      msg.textContent = "";
      try {
        await loginAdmin(pw.value);
        // On success, notes.js shows the upload panel automatically
        pw.value = "";
        form.classList.add("hidden");
      } catch (err) {
        msg.textContent = errorText(err);
      }
    });
  </script>
</body>
</html>
```

A.C style.css (part 1 of 4)

```
/* ============================================
   Dead-Drop theme
   To change the theme, only edit the values inside :root
   ============================================ */
:root {
  --bg: #050805;
  --panel: rgba(8, 16, 8, 0.85);
  --text: #b6ffb6;
  --muted: #5f8f5f;
  --accent: #00ff66;
  --danger: #ff4d4d;
  --border: #12401f;
  --font: "Consolas", "Courier New", monospace;
}

* { box-sizing: border-box; margin: 0; padding: 0; }

html, body { min-height: 100%; }

body {
  background: var(--bg);
  color: var(--text);
  font-family: var(--font);
  line-height: 1.6;
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

/* ---------- Matrix background ---------- */
#matrix {
  position: fixed;
  inset: 0;
  width: 100%;
  height: 100%;
  z-index: 0;
  opacity: 0.35;
}

.wrap {
  position: relative;
  z-index: 1;
  flex: 1;
  width: 100%;
  max-width: 820px;
  margin: 0 auto;
  padding: 40px 20px;
}

.center {
  display: flex;
  align-items: center;
  justify-content: center;
}

/* ---------- Terminal box ---------- */
.terminal {
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 24px;
  box-shadow: 0 0 25px rgba(0, 255, 102, 0.12);
  width: 100%;
}

.terminal-bar {
```

A.C style.css (part 2 of 4)

```
  color: var(--muted);
  font-size: 0.8rem;
  margin-bottom: 14px;
  border-bottom: 1px solid var(--border);
  padding-bottom: 8px;
}

.line { margin: 4px 0; min-height: 1.6em; }
.prompt { color: var(--accent); }

.cursor {
  display: inline-block;
  width: 9px;
  height: 1em;
  background: var(--accent);
  vertical-align: text-bottom;
  animation: blink 1s steps(1) infinite;
}
@keyframes blink { 50% { opacity: 0; } }

h1 {
  color: var(--accent);
  font-size: 2rem;
  letter-spacing: 3px;
  text-shadow: 0 0 10px rgba(0, 255, 102, 0.6);
}

h2 { color: var(--accent); font-size: 1.2rem; margin-bottom: 12px; }

/* ---------- Inputs & buttons ---------- */
input[type="password"],
input[type="email"],
input[type="text"],
input[type="search"] {
  width: 100%;
  background: #000;
  color: var(--accent);
  border: 1px solid var(--border);
  border-radius: 4px;
  padding: 10px 12px;
  font-family: var(--font);
  font-size: 1rem;
  outline: none;
}
input:focus {
  border-color: var(--accent);
  box-shadow: 0 0 8px rgba(0, 255, 102, 0.35);
}

.btn {
  display: inline-block;
  background: transparent;
  color: var(--accent);
  border: 1px solid var(--accent);
  border-radius: 4px;
  padding: 9px 18px;
  font-family: var(--font);
  font-size: 0.95rem;
  cursor: pointer;
  text-decoration: none;
  transition: 0.2s;
}
.btn:hover {
  background: var(--accent);
  color: #000;
  box-shadow: 0 0 14px rgba(0, 255, 102, 0.6);
```

A.C style.css (part 3 of 4)

```
}
.btn:disabled { opacity: 0.4; cursor: not-allowed; }

.row { display: flex; gap: 10px; margin-top: 14px; flex-wrap: wrap; }
.row input { flex: 1; min-width: 180px; }

.error { color: var(--danger); margin-top: 10px; min-height: 1.4em; }
.hint { color: var(--muted); font-size: 0.85rem; margin-top: 10px; }
.hidden { display: none !important; }

/* ---------- Notes page ---------- */
.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
  margin-bottom: 20px;
}

.list { margin-top: 18px; }

.file {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 6px;
  padding: 12px 14px;
  margin-bottom: 10px;
}
.file:hover { border-color: var(--accent); }
.file-name { word-break: break-word; }
.file-meta { color: var(--muted); font-size: 0.8rem; }

.crumbs {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 6px;
  margin-top: 14px;
  color: var(--muted);
}

.admin-panel {
  margin-top: 24px;
  border: 1px dashed var(--accent);
  border-radius: 6px;
  padding: 16px;
}

/* ---------- Footer ---------- */
.site-footer {
  position: relative;
  z-index: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  text-align: center;
  color: var(--muted);
  font-family: var(--font);
  font-size: 0.85rem;
  padding: 20px 10px 16px;
  border-top: 1px solid var(--border);
```

A.C style.css (part 4 of 4)

```
  background: linear-gradient(to bottom, transparent, var(--panel));
  text-shadow: 0 0 6px rgba(0, 255, 102, 0.25);
}
.site-footer b { color: var(--accent); }

.quote-ar {
  font-family: "Traditional Arabic", "Amiri", "Scheherazade New", serif;
  font-size: 1.5rem;
  color: var(--accent);
  margin: 0 auto 10px;
  line-height: 2;
  text-align: center;
  direction: rtl;
  text-shadow: 0 0 10px rgba(0, 255, 102, 0.6);
}

.quote-bn {
  font-family: var(--font);
  font-size: 0.95rem;
  color: var(--text);
  margin: 0 auto 12px;
  line-height: 1.7;
  text-align: center;
  opacity: 0.85;
}

@media (max-width: 520px) {
  h1 { font-size: 1.5rem; }
  .file { flex-direction: column; align-items: flex-start; }
  .quote-ar { font-size: 1.2rem; }
}
```

A.D config.js

```
// ============================================
//  Dead-Drop config
// ============================================

export const SITE_NAME = "Dead-Drop";
export const TEAM_NAME = "BreakByte";

// PDF files are stored in the notes/ folder of the GitHub repo
export const GITHUB = {
  user: "YOUR_GITHUB_USERNAME",
  repo: "Dead-Drop",
  branch: "main",
  folder: "notes",
};

// Firebase Authentication (login only)
export const FIREBASE = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT_ID.firebaseapp.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT_ID.firebasestorage.app",
  messagingSenderId: "YOUR_SENDER_ID",
  appId: "YOUR_APP_ID",
};

// Shared viewer account for friends
export const VIEWER_EMAIL = "YOUR_VIEWER_EMAIL";

// Your admin account
export const ADMIN_EMAIL = "YOUR_ADMIN_EMAIL";
```

A.E auth.js

```
// ============================================
//  Firebase Authentication (login only)
//  Firebase loads directly from the CDN, no npm required
// ============================================
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-app.js";
import {
  getAuth,
  signInWithEmailAndPassword,
  signOut,
  onAuthStateChanged,
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-auth.js";
import { FIREBASE, VIEWER_EMAIL, ADMIN_EMAIL } from "./config.js";

const app = initializeApp(FIREBASE);
const auth = getAuth(app);

// Viewer login: only a password is needed, the email is stored in code
export function loginViewer(password) {
  return signInWithEmailAndPassword(auth, VIEWER_EMAIL, password);
}

// Admin login: the admin email is stored in code, only the password is typed
export function loginAdmin(password) {
  return signInWithEmailAndPassword(auth, ADMIN_EMAIL, password);
}

export function logout() {
  return signOut(auth);
}

// Calls callback(user) whenever the login state changes (user is null if logged out)
export function watchAuth(callback) {
  return onAuthStateChanged(auth, callback);
}

export function isAdmin(user) {
  return !!user && user.email === ADMIN_EMAIL;
}

// Converts Firebase error codes into simple, user-friendly messages
export function errorText(err) {
  switch (err && err.code) {
    case "auth/invalid-credential":
    case "auth/wrong-password":
    case "auth/user-not-found":
    case "auth/invalid-email":
      return "ACCESS DENIED: Wrong password";
    case "auth/too-many-requests":
      return "Too many failed attempts. Please try again later.";
    case "auth/network-request-failed":
      return "Please check your internet connection.";
    case "auth/invalid-api-key":
      return "The apiKey in config.js is invalid.";
    case "auth/unauthorized-domain":
      return "Add raito-del.github.io to Firebase Authorized domains.";
    default:
      return "Error: " + (err && err.code ? err.code : "Unknown error");
  }
}
```

A.F github.js

```
// Everything for creating folders and uploading PDFs to GitHub lives here.
import { GITHUB } from "./config.js";

const TOKEN_KEY = "dd_gh_token";
export const getToken = () => localStorage.getItem(TOKEN_KEY) || "";
export const saveToken = (t) => localStorage.setItem(TOKEN_KEY, t.trim());
export const clearToken = () => localStorage.removeItem(TOKEN_KEY);

const API = `https://api.github.com/repos/${GITHUB.user}/${GITHUB.repo}/contents`;

const encPath = (p) => p.split("/").map(encodeURIComponent).join("/");

export function fileToBase64(file) {
  return new Promise((resolve, reject) => {
    const r = new FileReader();
    r.onload = () => resolve(r.result.split(",")[1]);
    r.onerror = () => reject(new Error("Could not read the file"));
    r.readAsDataURL(file);
  });
}

async function putFile(repoPath, base64Content, message) {
  const token = getToken();
  if (!token) throw new Error("Please save a GitHub token first");

  const res = await fetch(`${API}/${encPath(repoPath)}`, {
    method: "PUT",
    headers: {
      Authorization: `Bearer ${token}`,
      Accept: "application/vnd.github+json",
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      message,
      content: base64Content,
      branch: GITHUB.branch,
    }),
  });

  if (res.status === 401 || res.status === 403) {
    throw new Error("Token is invalid or does not have permission");
  }
  if (res.status === 422 || res.status === 409) {
    throw new Error("A folder or file with this name already exists");
  }
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(body.message || "HTTP " + res.status);
  }
}

// A folder is created by adding an empty .gitkeep file inside it (GitHub does not store empty folders)
export async function createFolder(rel) {
  await putFile(`${GITHUB.folder}/${rel}/.gitkeep`, "", `Create folder ${rel}`);
}

export async function uploadPdf(file, rel) {
  const base64 = await fileToBase64(file);
  await putFile(`${GITHUB.folder}/${rel}`, base64, `Upload ${rel}`);
}
```

A.G notes.js (part 1 of 6)

```
// ============================================
//  Notes page: login check, folder view, search, admin (folders + upload)
// ============================================
import { GITHUB } from "./config.js";
import { watchAuth, logout, isAdmin } from "./auth.js";
import { getToken, saveToken, clearToken, createFolder, uploadPdf } from "./github.js";

const $ = (id) => document.getElementById(id);
const listEl = $("list");
const searchEl = $("search");
const statusEl = $("status");
const content = $("content");
const adminPanel = $("admin-panel");
const logoutBtn = $("logout");

const crumbsEl = document.createElement("div");
crumbsEl.className = "crumbs";
listEl.before(crumbsEl);

let allFiles = [];     // { name, path, folders[], size }
let allFolders = [];   // [["first-year"], ["first-year", "cp-2"]]
let currentPath = [];  // the folder the user is currently in
let isAdminUser = false;
let adminMsg = null;

// ---------- helper ----------
const el = (tag, props = {}, ...kids) => {
  const n = document.createElement(tag);
  Object.assign(n, props);
  n.append(...kids);
  return n;
};

// Remove characters from folder/file names that GitHub does not allow
function cleanName(s) {
  return s
    .replace(/[\/\\:*?"<>|]/g, "")
    .replace(/\s+/g, " ")
    .replace(/^\.+/, "")
    .trim()
    .slice(0, 120);
}

// Is segs inside path (or exactly at it)?
const under = (segs, path) => path.every((p, i) => segs[i] === p);

function sizeText(bytes) {
  if (!bytes) return "";
  return bytes > 1048576
    ? (bytes / 1048576).toFixed(1) + " MB"
    : Math.max(1, Math.round(bytes / 1024)) + " KB";
}

function rawUrl(relPath) {
  const encoded = [GITHUB.folder, ...relPath.split("/")].map(encodeURIComponent).join("/");
  return `https://raw.githubusercontent.com/${GITHUB.user}/${GITHUB.repo}/${GITHUB.branch}/${encoded}`;
}

function showMsg(text, ok = false) {
  if (!adminMsg) return;
  adminMsg.textContent = text;
  adminMsg.className = ok ? "hint" : "error";
}

async function run(btn, fn) {
  btn.disabled = true;
```

A.G notes.js (part 2 of 6)

```
  showMsg("Working...", true);
  try {
    await fn();
  } catch (err) {
    showMsg("Error: " + err.message);
  } finally {
    btn.disabled = false;
  }
}

// ---------- login guard ----------
watchAuth((user) => {
  if (!user) { location.replace("index.html"); return; }
  content.classList.remove("hidden");
  isAdminUser = isAdmin(user);
  renderAdmin();
  loadFiles();
});

logoutBtn.addEventListener("click", async () => {
  await logout();
  location.replace("index.html");
});

// ---------- load all files and folders from GitHub ----------
async function loadFiles() {
  statusEl.textContent = "Loading...";
  const api =
      `https://api.github.com/repos/${GITHUB.user}/${GITHUB.repo}/git/trees/${GITHUB.branch}?recursive=1`;
  try {
    const res = await fetch(api, { cache: "no-store" });
    if (!res.ok) throw new Error("HTTP " + res.status);
    const data = await res.json();
    const prefix = GITHUB.folder + "/";

    allFiles = [];
    const folderSet = new Map();

    for (const item of data.tree) {
      if (!item.path.startsWith(prefix)) continue;
      const rel = item.path.slice(prefix.length);
      const parts = rel.split("/");

      if (item.type === "tree") {
        folderSet.set(rel, parts);
        continue;
      }
      if (item.type !== "blob") continue;

      for (let i = 1; i < parts.length; i++) {
        const p = parts.slice(0, i);
        folderSet.set(p.join("/"), p);
      }
      if (/\.pdf$/i.test(rel)) {
        allFiles.push({
          name: parts[parts.length - 1],
          path: rel,
          folders: parts.slice(0, -1),
          size: item.size,
        });
      }
    }

    allFolders = [...folderSet.values()];
    allFiles.sort((a, b) => a.path.localeCompare(b.path));
    render();
```

A.G notes.js (part 3 of 6)

```
    renderAdmin();
  } catch (err) {
    statusEl.textContent = "Could not load the list: " + err.message;
  }
}

// ---------- render ----------
function render() {
  const q = searchEl.value.trim().toLowerCase();
  listEl.innerHTML = "";
  closeViewer();

  if (q) {
    crumbsEl.textContent = "Search results (from all folders)";
    const matches = allFiles.filter(
      (f) => f.name.toLowerCase().includes(q) || f.path.toLowerCase().includes(q)
    );
    statusEl.textContent = matches.length ? "" : "Nothing found.";
    matches.forEach((f) => listEl.append(fileRow(f, f.folders.join(" / "))));
    return;
  }

  renderCrumbs();
  const depth = currentPath.length;

  const subfolders = [
    ...new Set(
      allFolders
        .filter((s) => s.length > depth && under(s, currentPath))
        .map((s) => s[depth])
    ),
  ].sort((a, b) => a.localeCompare(b));

  const files = allFiles.filter(
    (f) => f.folders.length === depth && under(f.folders, currentPath)
  );

  subfolders.forEach((name) => {
    const count = allFiles.filter(
      (f) => f.folders.length > depth && under(f.folders, currentPath) && f.folders[depth] === name
    ).length;
    listEl.append(folderRow(name, count));
  });
  files.forEach((f) => listEl.append(fileRow(f, "")));

  if (!allFiles.length && !subfolders.length) statusEl.textContent = "Nothing in notes yet.";
  else if (!subfolders.length && !files.length) statusEl.textContent = "This folder is empty for now.";
  else statusEl.textContent = "";
}

function renderCrumbs() {
  crumbsEl.replaceChildren();
  const make = (label, path) => {
    const b = el("button", { className: "btn ghost", textContent: label });
    b.addEventListener("click", () => {
      currentPath = path;
      searchEl.value = "";
      render();
      renderAdmin();
    });
    return b;
  };
  crumbsEl.append(make("notes", []));
  currentPath.forEach((seg, i) => {
    crumbsEl.append(" / ", make(seg, currentPath.slice(0, i + 1)));
  });
```

A.G notes.js (part 4 of 6)

```
}

function folderRow(name, count) {
  const open = el("button", { className: "btn", textContent: "Open" });
  open.addEventListener("click", () => {
    currentPath = [...currentPath, name];
    searchEl.value = "";
    render();
    renderAdmin();
  });

  const info = el("div", {},
    el("div", { className: "file-name", textContent: name }),
    el("div", { className: "file-meta", textContent: "Folder · " + count + " PDF(s)" })
  );
  return el("div", { className: "file" }, info, open);
}

function fileRow(f, label) {
  const read = el("button", { className: "btn", textContent: "Read", type: "button" });
  read.addEventListener("click", () => openViewer(f));

  const dl = el("a", { className: "btn", textContent: "Download" });
  dl.href = rawUrl(f.path);
  dl.target = "_blank";
  dl.rel = "noopener";
  dl.setAttribute("download", f.name);

  const meta = [label, sizeText(f.size)].filter(Boolean).join(" · ");
  const info = el("div", {},
    el("div", { className: "file-name", textContent: f.name.replace(/\.pdf$/i, "") }),
    el("div", { className: "file-meta", textContent: meta })
  );
  const actions = el("div", { style: "display:flex; gap:8px; flex-wrap:wrap;" }, read, dl);
  return el("div", { className: "file" }, info, actions);
}

// ---------- PDF viewer (shows the PDF inside the site) ----------
function closeViewer() {
  const old = document.getElementById("pdf-viewer");
  if (old) old.remove();
}

function openViewer(f) {
  closeViewer();

  const box = el("div", { id: "pdf-viewer", className: "admin-panel" });
  const close = el("button", { className: "btn ghost", textContent: "Close", type: "button" });
  close.addEventListener("click", closeViewer);

  // pdf.js viewer: it can display PDFs from GitHub raw links
  const viewerUrl =
    "https://mozilla.github.io/pdf.js/web/viewer.html?file=" + encodeURIComponent(rawUrl(f.path));
  const frame = el("iframe", { src: viewerUrl });
  frame.style.cssText = "width:100%; height:70vh; border:0; background:#000; margin-top:12px;";

  box.append(
    el("div", { style: "display:flex; justify-content:space-between; align-items:center; gap:10px;" },
      el("h2", { textContent: f.name.replace(/\.pdf$/i, ""), style: "margin:0;" }),
      close
    ),
    frame
  );

  listEl.after(box);
  box.scrollIntoView({ behavior: "smooth" });
```

A.G notes.js (part 5 of 6)

```
}

// ---------- admin panel ----------
function renderAdmin() {
  if (!isAdminUser) return;
  adminPanel.classList.remove("hidden");
  adminPanel.replaceChildren();

  const where = "notes" + (currentPath.length ? " / " + currentPath.join(" / ") : "");
  adminPanel.append(
    el("h2", { textContent: "admin: folders & PDFs" }),
    el("div", { className: "hint", textContent: "You are in: " + where })
  );

  if (!getToken()) {
    const tok = el("input", { type: "password", placeholder: "GitHub token (enter once)", autocomplete:
        "off" });
    const save = el("button", { className: "btn", textContent: "Save", type: "button" });
    save.onclick = () => {
      if (!tok.value.trim()) return showMsg("Token is empty");
      saveToken(tok.value);
      renderAdmin();
    };
    adminPanel.append(el("div", { className: "row" }, tok, save));
  } else {
    const clearBtn = el("button", { className: "btn ghost", textContent: "Delete token", type: "button" });
    clearBtn.onclick = () => { clearToken(); renderAdmin(); };
    adminPanel.append(
      el("div", { className: "hint", textContent: "GitHub token is saved in this browser." }),
      clearBtn
    );

    // New folder
    const folderIn = el("input", { type: "text", placeholder: "New folder name" });
    const folderBtn = el("button", { className: "btn", textContent: "Create folder", type: "button" });
    folderBtn.onclick = () => run(folderBtn, async () => {
      const name = cleanName(folderIn.value);
      if (!name) throw new Error("Enter a folder name");
      await createFolder([...currentPath, name].join("/"));
      currentPath = [...currentPath, name];
      await loadFiles();
      showMsg("Folder created: " + name, true);
    });
    adminPanel.append(
      el("h2", { textContent: "New folder" }),
      el("div", { className: "row" }, folderIn, folderBtn)
    );

    // PDF upload
    const pdfIn = el("input", { type: "file", accept: "application/pdf" });
    const pdfName = el("input", { type: "text", placeholder: "PDF name (optional)" });
    const upBtn = el("button", { className: "btn", textContent: "Upload PDF", type: "button" });
    upBtn.onclick = () => run(upBtn, async () => {
      const file = pdfIn.files[0];
      if (!file) throw new Error("Select a PDF first");
      if (file.size > 95 * 1024 * 1024) throw new Error("File is too large (max ~95MB)");

      const base = cleanName(pdfName.value || file.name).replace(/\.pdf$/i, "").trim();
      if (!base) throw new Error("Enter a PDF name");
      const name = base + ".pdf";

      await uploadPdf(file, [...currentPath, name].join("/"));
      pdfIn.value = "";
      pdfName.value = "";
      await loadFiles();
      showMsg("Uploaded: " + name, true);
```

A.G notes.js (part 6 of 6)

```
    });
    adminPanel.append(
      el("h2", { textContent: "Upload PDF to this folder" }),
      el("div", { className: "row" }, pdfIn),
      el("div", { className: "row" }, pdfName, upBtn)
    );
  }

  adminMsg = el("div", { className: "hint" });
  adminPanel.append(adminMsg);
}

searchEl.addEventListener("input", render);
```

A.H matrix.js

```
// ============================================
//  Matrix rain background
//  The HTML must include <canvas id="matrix"></canvas>
// ============================================

const canvas = document.getElementById("matrix");

if (canvas) {
  const ctx = canvas.getContext("2d");
  const chars = "01[...]ABCDEFGHIJKLMNOPQRSTUVWXYZ<>/{}[]#$%&*+=";
  const size = 16;
  let drops = [];

  function resize() {
    canvas.width = window.innerWidth;
    canvas.height = window.innerHeight;
    const columns = Math.floor(canvas.width / size);
    drops = Array.from({ length: columns }, () => Math.random() * -50);
  }

  function draw() {
    // The previous frame slowly fades out, which creates the trailing effect
    ctx.fillStyle = "rgba(5, 8, 5, 0.12)";
    ctx.fillRect(0, 0, canvas.width, canvas.height);

    ctx.fillStyle = "#00ff66";
    ctx.font = size + "px monospace";

    for (let i = 0; i < drops.length; i++) {
      const text = chars.charAt(Math.floor(Math.random() * chars.length));
      ctx.fillText(text, i * size, drops[i] * size);

      if (drops[i] * size > canvas.height && Math.random() > 0.975) {
        drops[i] = 0;
      }
      drops[i]++;
    }
  }

  resize();
  window.addEventListener("resize", resize);
  setInterval(draw, 50);
}
```

A.I typing.js

```
// ============================================
//  Terminal typing effect
//  index.html must have id="typing-lines" and id="gate"
// ============================================
import { SITE_NAME } from "./config.js";

const box = document.getElementById("typing-lines");
const gate = document.getElementById("gate");

// Change these lines to whatever you like
const lines = [
  "> booting " + SITE_NAME.toLowerCase() + " ...",
  "> establishing secure connection ...",
  "> encryption layer: OK",
  "> access restricted. authentication required.",
];

const SPEED = 35;   // milliseconds per character
const PAUSE = 350;  // pause after each line

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

async function run() {
  if (!box) return;

  for (const text of lines) {
    const line = document.createElement("div");
    line.className = "line prompt";
    box.appendChild(line);

    for (const ch of text) {
      line.textContent += ch;
      await sleep(SPEED);
    }
    await sleep(PAUSE);
  }

  const cursor = document.createElement("span");
  cursor.className = "cursor";
  box.lastChild.appendChild(cursor);

  if (gate) {
    gate.classList.remove("hidden");
    const input = gate.querySelector("input");
    if (input) input.focus();
  }
}

run();
```

A.J README.md

```
# Dead Drop

A private notes vault made by Team BreakByte.

## Overview
Dead Drop is a password-protected website where members can browse and download study notes. Notes are
    organized into folders, and the site owner can create folders and upload PDFs directly from the website.

## Features
- Password-protected access for viewers
- Separate admin login for managing content
- Folder-based note organization
- Search across all folders
- PDF download for every visitor

## Admin Guide
1. Open the notes page and log in as admin.
2. Save the access token once in the admin panel. It is stored only in the current browser.
3. Create folders and open them to go deeper.
4. Upload PDFs to the folder you are currently in.
5. Use the search box to find any file across all folders.

## Notes
- The site must be opened through a web server or the hosted link, not by opening the file directly on
    your computer.
- Anyone who can view the repository can also view the notes, so keep sensitive material out of it.
- Keep the access token private. Never share it, post it, or save it in shared files.

## Team
Made by Team BreakByte.
```